CAPSTONE PROJECT

Utility Billing System

A SECURE FULL-STACK MICROSERVICES APPLICATION FOR UTILITY MANAGEMENT

BUILT WITH SPRING BOOT ANGULAR, AND MONGODB

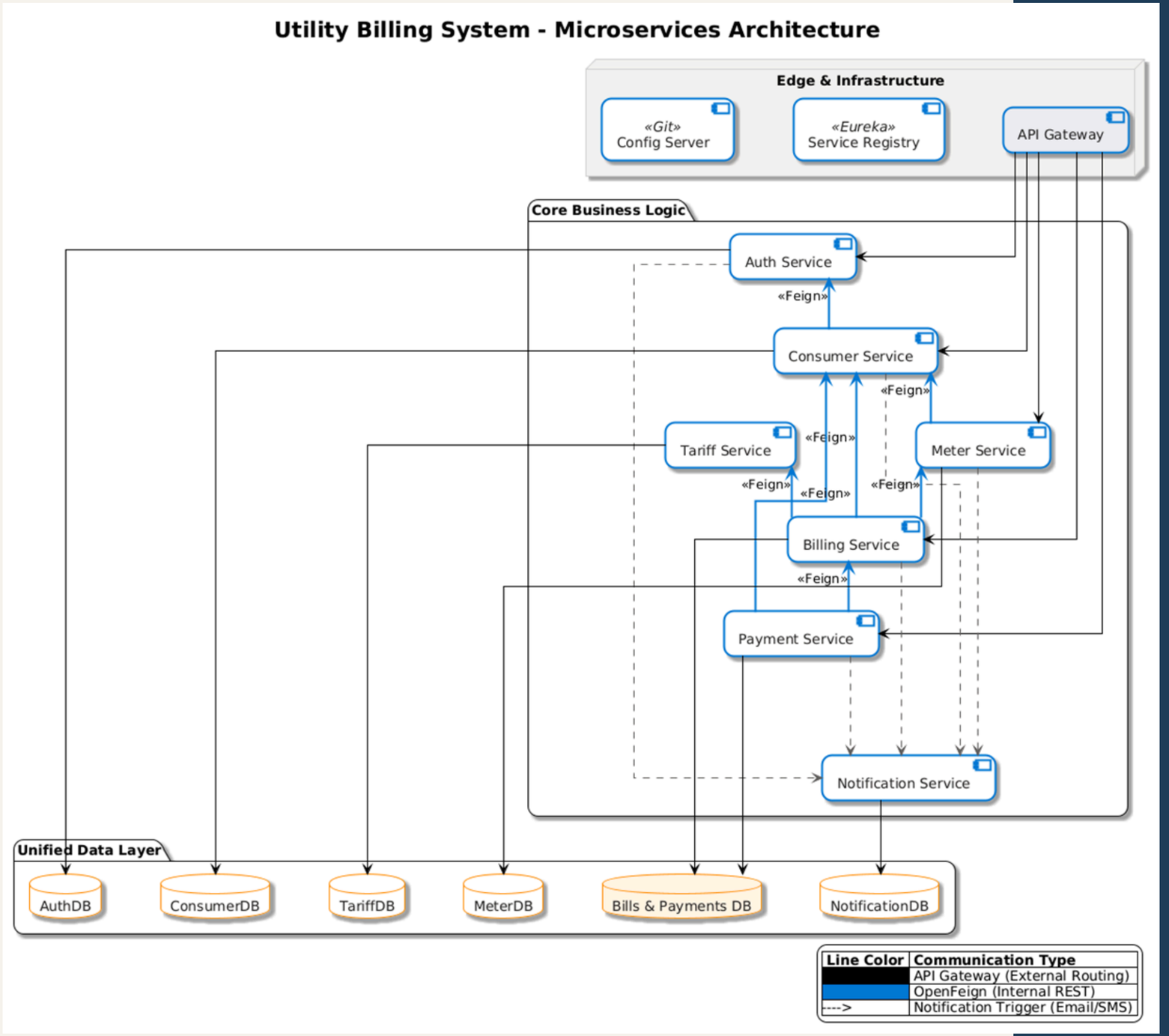
Project Overview

This system is a digital platform that helps manage household services like electricity and gas by tracking usage through meter readings, automatically calculating monthly bills, and providing an easy way for people to pay their dues online or at a counter.

Key Features:

- Centralized Control: Unified management of Electricity, Gas, and Water utilities.
- Billing Accuracy: Automated calculation using dynamic Multi-Slab Tariff logic.
- Role-Based Access: Secure dashboards for Admin, Billing, Accounts, and Consumers.
- Seamless Payments: Integrated OTP-verified online payments and offline recording.

System Architecture



System Architecture

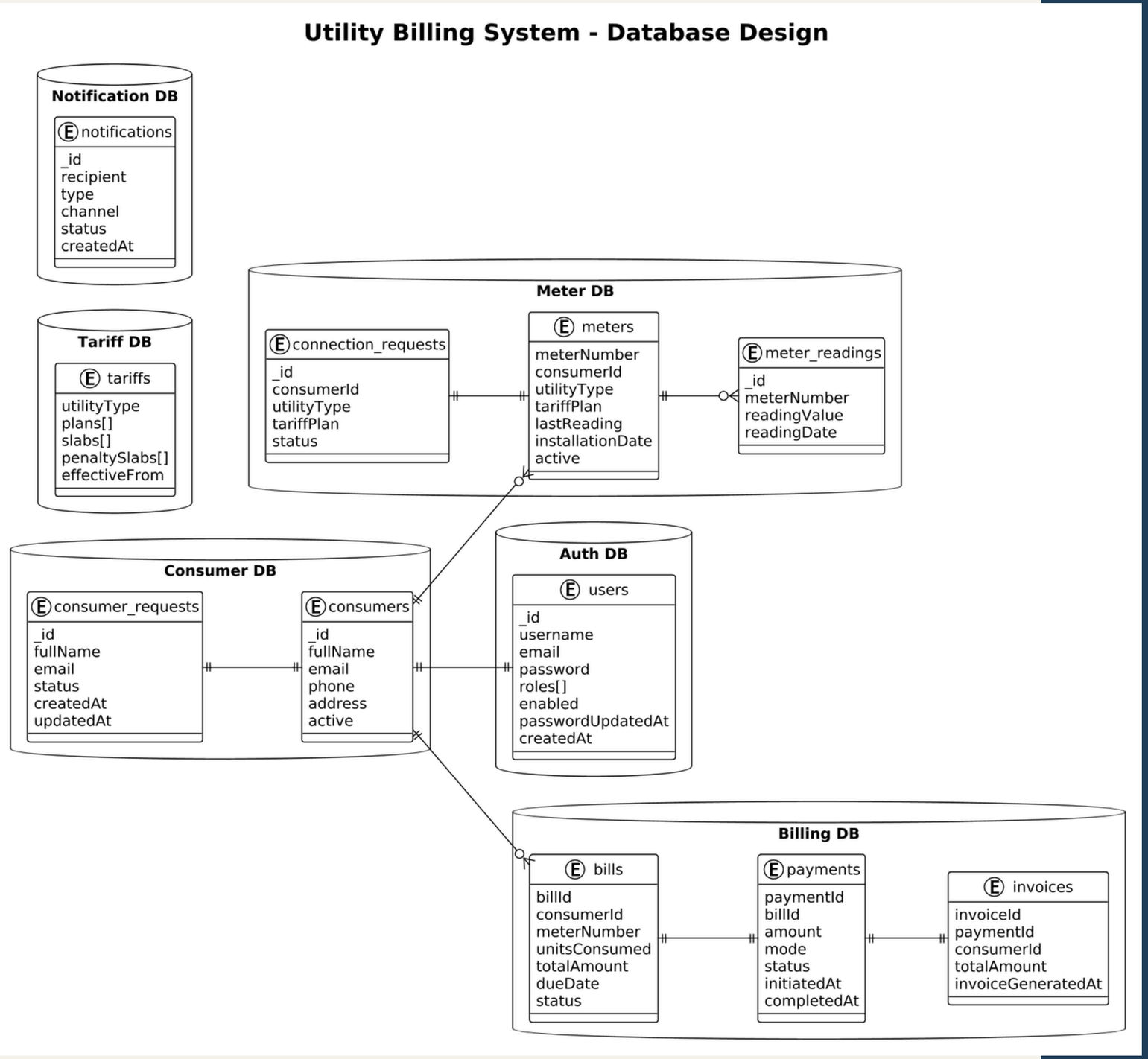
Infrastructure Components:

- API Gateway: Single entry point for routing, JWT validation.
- Service Registry (Eureka): Dynamically tracks all active microservice instances.
- Config Server: Centralized, Git-backed management for service properties.

Microservices Breakdown

- Auth Service: Manages user identities and security tokens.
- Consumer Service: Handles the lifecycle of ConsumerRequest and consumer profiles.
- Meter Service: Manages ConnectionRequests, physical meter mapping, and MeterReadings.
- Tariff Service: CRUD operations for utility plans and multi-slab rate logic.
- Billing Service: The engine that calculates charges based on consumption and tariff slabs.
- Payment Service: Manages OTP-based online payments and offline receipts.
- Notification Service: Asynchronous engine for Email/SMS alerts.

Database Design



Database Design

- Each microservice owns its own database
- Prevents tight coupling between services
- Allows each service to scale independently
- Relationships are managed at the service layer, not database joins
- Financial data is immutable once finalized
- Updates are handled via status changes, not data modification

Why MongoDB?

- Data structures like tariffs, bills, and payments change over time
- MongoDB allows flexible schemas without frequent migrations
- Works naturally with microservices and document-based data

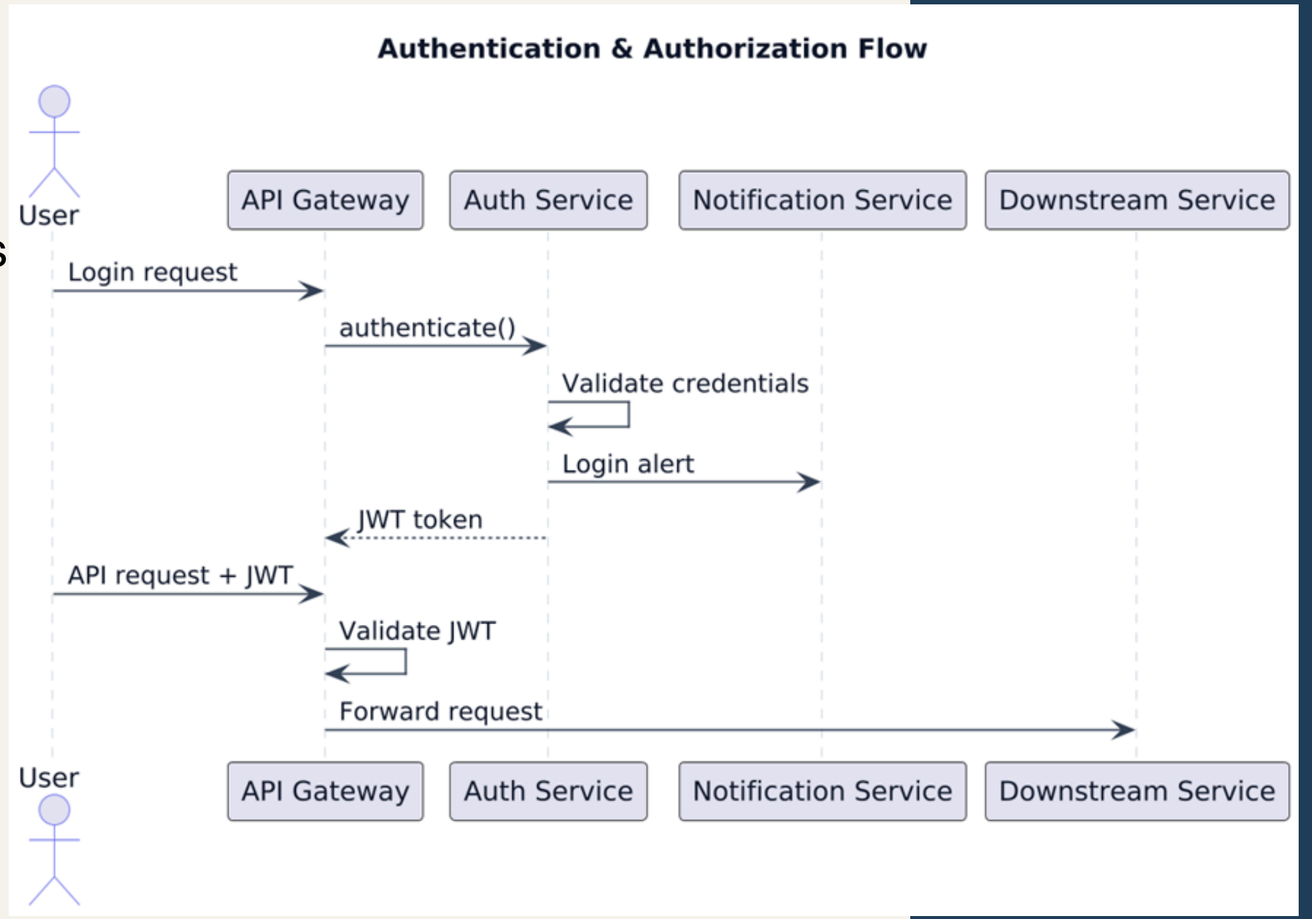
Functional Roles (Actors)

- Admin: Manages tariff plans, approves consumer/connection requests, and monitors system health.
- Billing Officer: Records periodic meter readings and triggers bill generation.
- Accounts Officer: Tracks outstanding balances and processes offline (walk-in) payments.
- Consumer: Views consumption history, pays bills online, and manages connection requests.

Security - Authentication & Authorization

A user authenticates using valid credentials, receives a JWT token, and accesses secured backend services through the API Gateway.

1. The user submits login credentials to the API Gateway.
2. The API Gateway forwards the authentication request to the Auth Service.
3. The Auth Service validates the user credentials against stored user data.
4. Upon successful authentication, the Auth Service generates a JWT token.
5. The JWT token is returned to the user through the API Gateway.
6. For subsequent requests, the user sends the JWT token along with the API request.
7. The API Gateway validates the JWT token and extracts user role information.
8. If valid, the request is forwarded to the appropriate downstream service.



Application Life Cycle

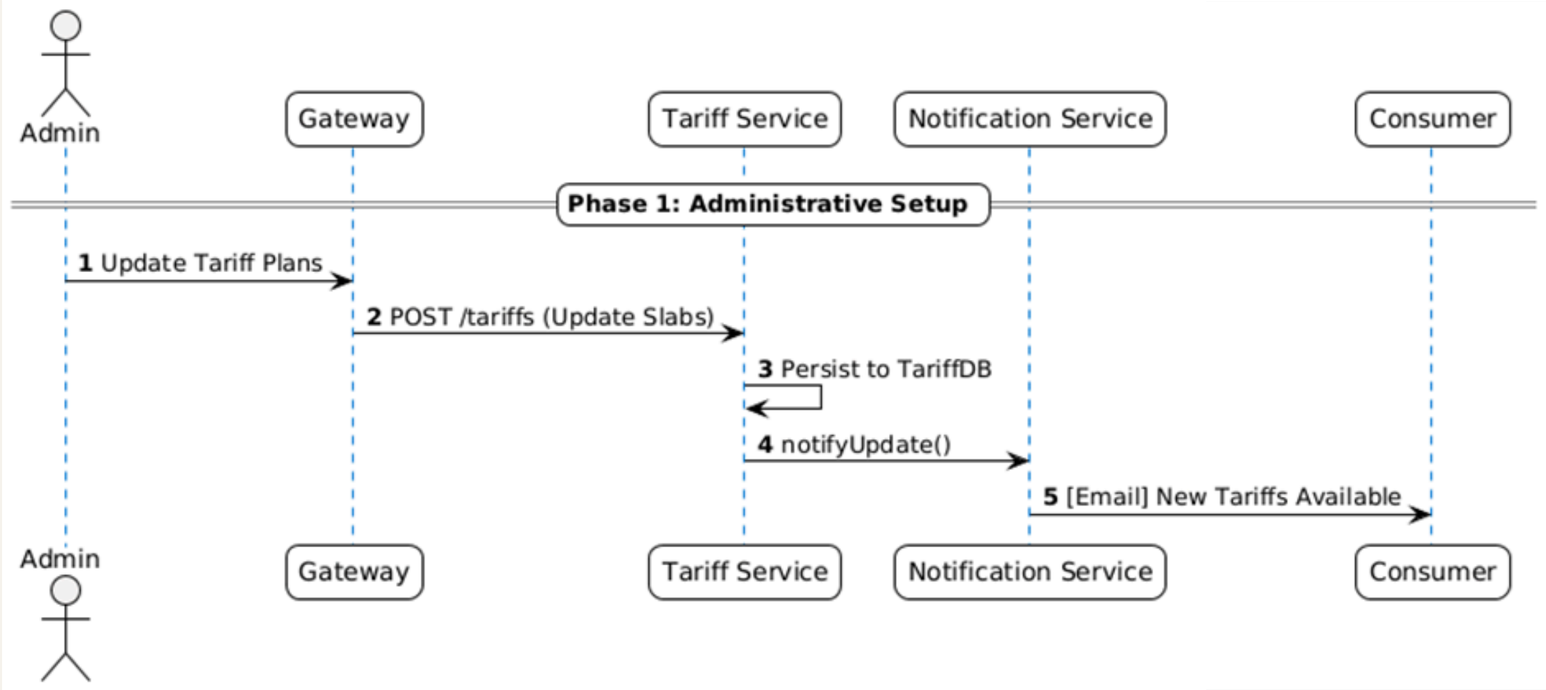
Phase 1: Administrative Setup

The administrator configures or updates tariff plans, which are stored centrally.

1.Admin updates tariff plans through the Gateway.

2.Gateway forwards the request to the Tariff Service.

3.Tariff Service saves the updated tariff configuration.

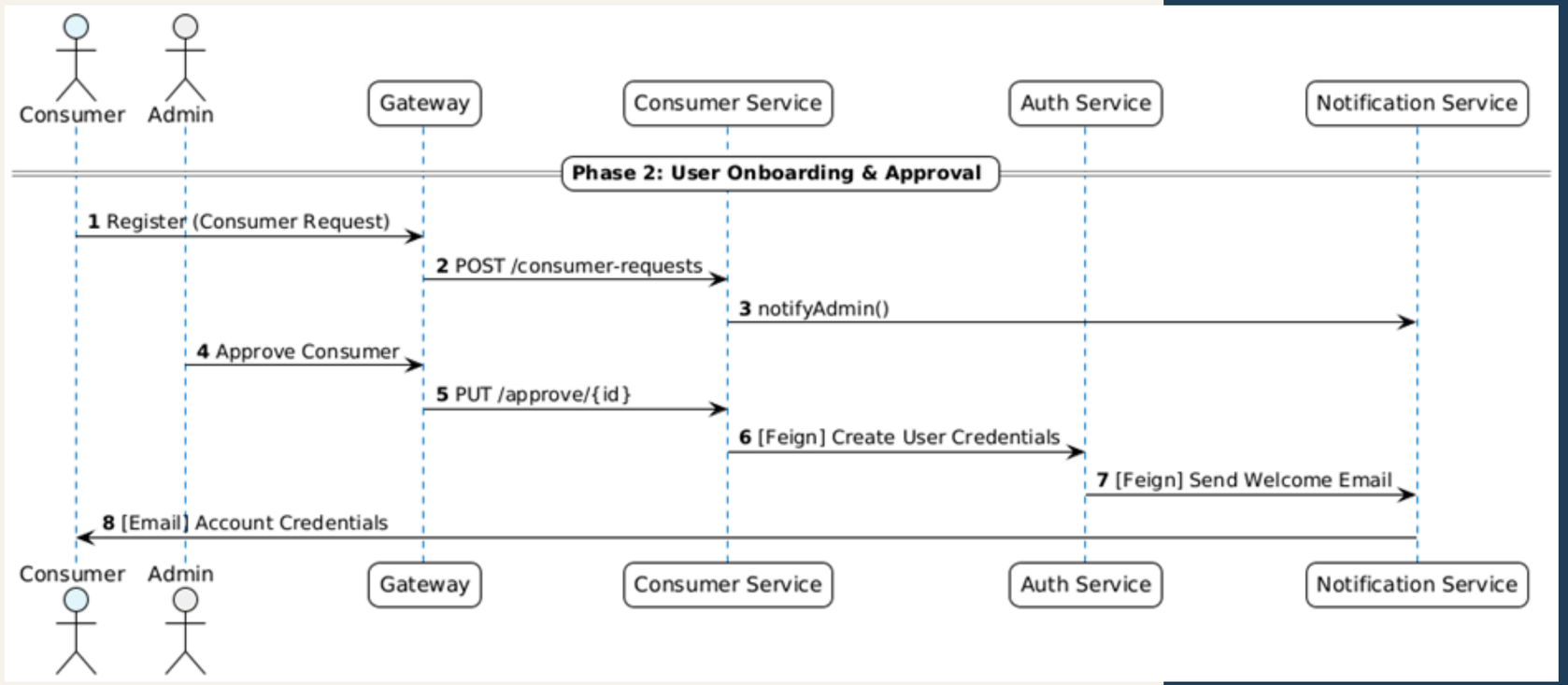


Application Life Cycle

Phase 2: User Onboarding & Approval

A consumer registers in the system and gets approved by the administrator to gain portal access.

1. Consumer submits a registration request through the Gateway.
2. Consumer Service creates a consumer request.
3. Admin approves the consumer request.
4. Auth Service creates login credentials.
5. Credentials are sent to the consumer via email.

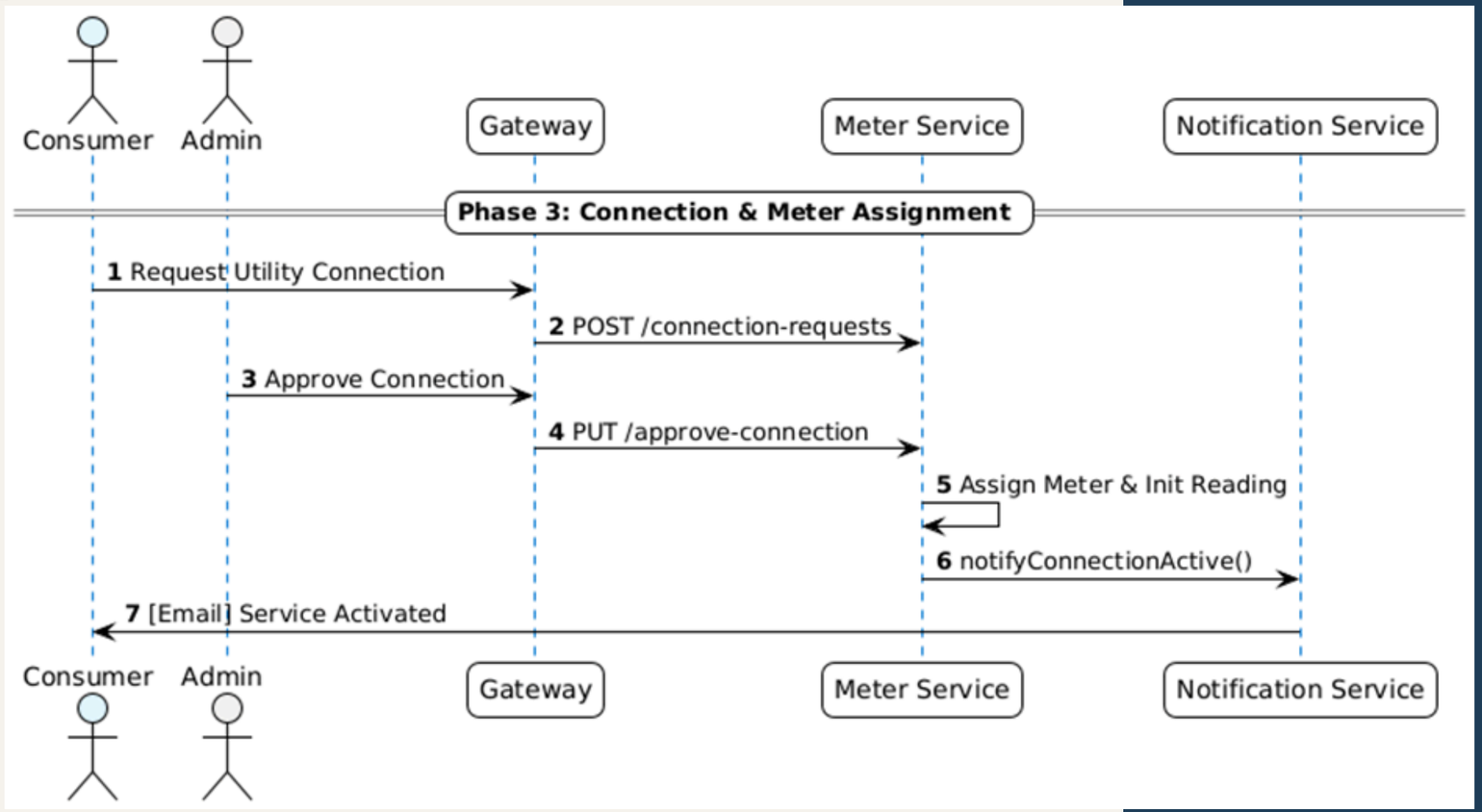


Application Life Cycle

Phase 3: Connection & Meter Assignment

An approved consumer requests a utility connection, which is validated by the administrator and a meter is assigned.

1. Consumer requests a utility connection.
2. Meter Service records the connection request.
3. Admin approves the connection request.
4. A meter is initialized.
5. Consumer is notified about service activation.

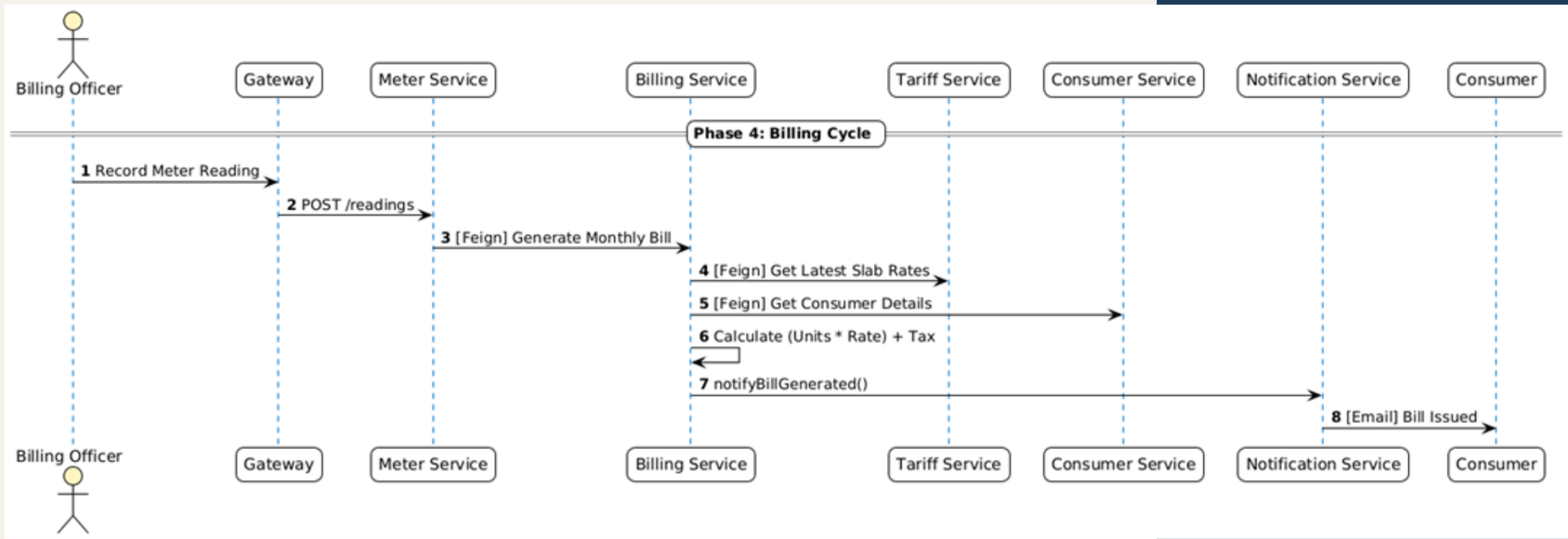


Application Life Cycle

Phase 4: Billing Cycle

Periodic meter readings are recorded by the billing officer and used to generate utility bills for consumers.

1. Billing Officer records meter readings.
2. Meter Service validates and forwards data for billing.
3. Billing Service retrieves tariff and consumer details.
4. Bill amount is calculated based on consumption and slabs.
5. Bill is generated and notified to the consumer.

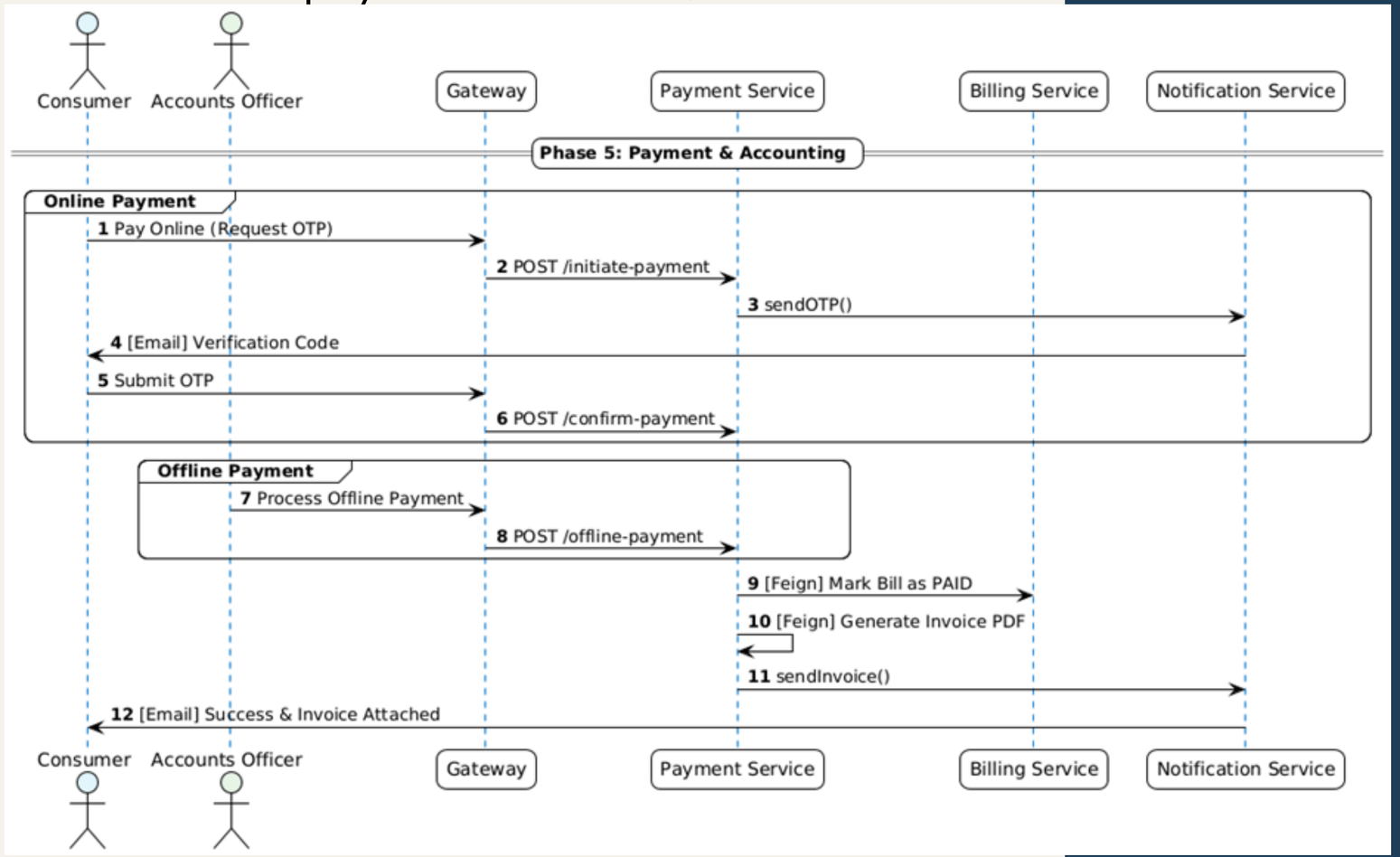


Application Life Cycle

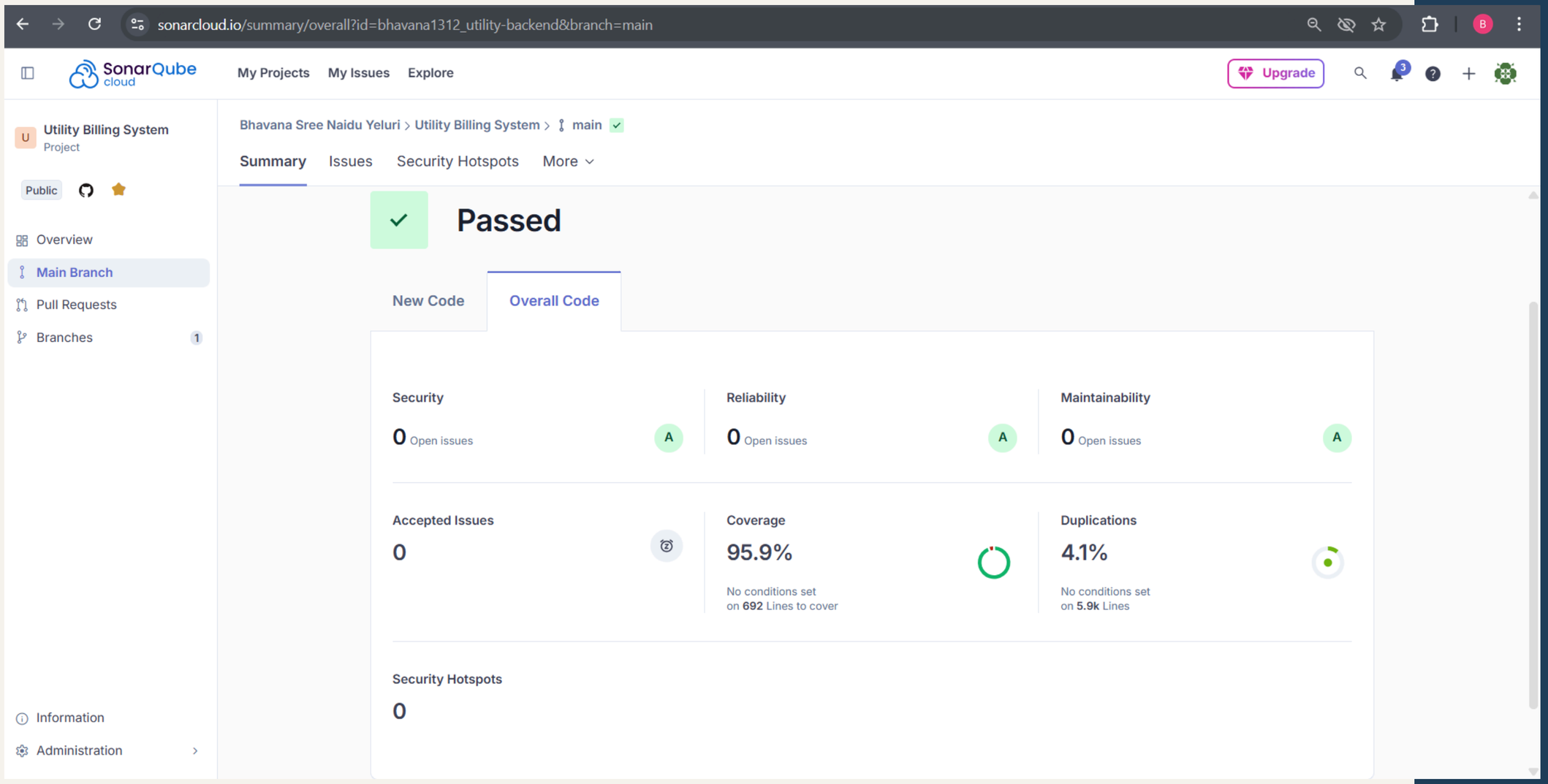
Phase 5: Payments & Accounting

Consumers settle bills through online or offline payment methods, and invoices are generated.

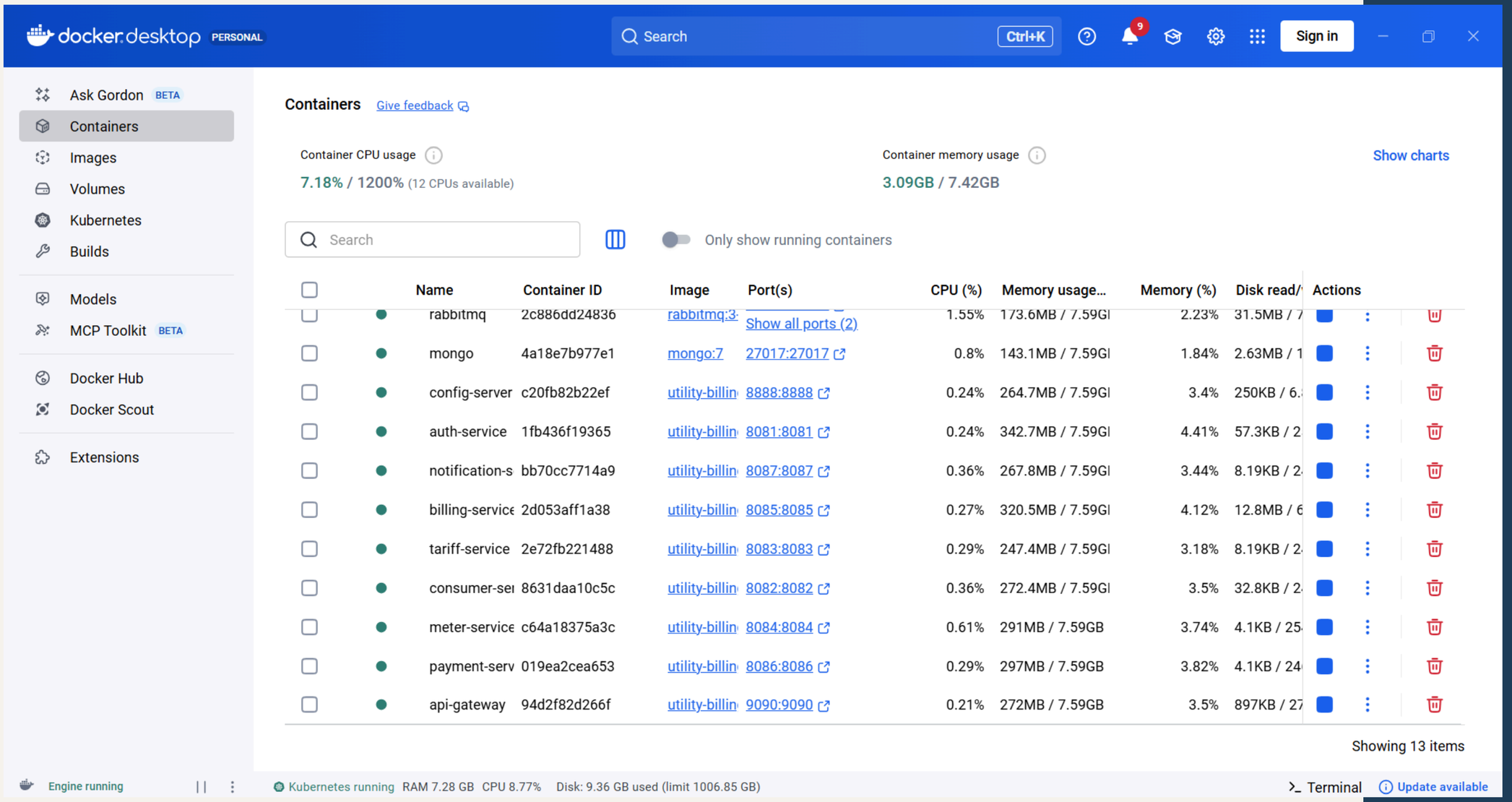
1. Consumer initiates online payment and requests OTP.
2. OTP is sent and verified before confirmation.
3. Accounts Officer can record offline payments.
4. Payment Service marks the bill as paid.
5. Invoice is generated and emailed to the consumer.



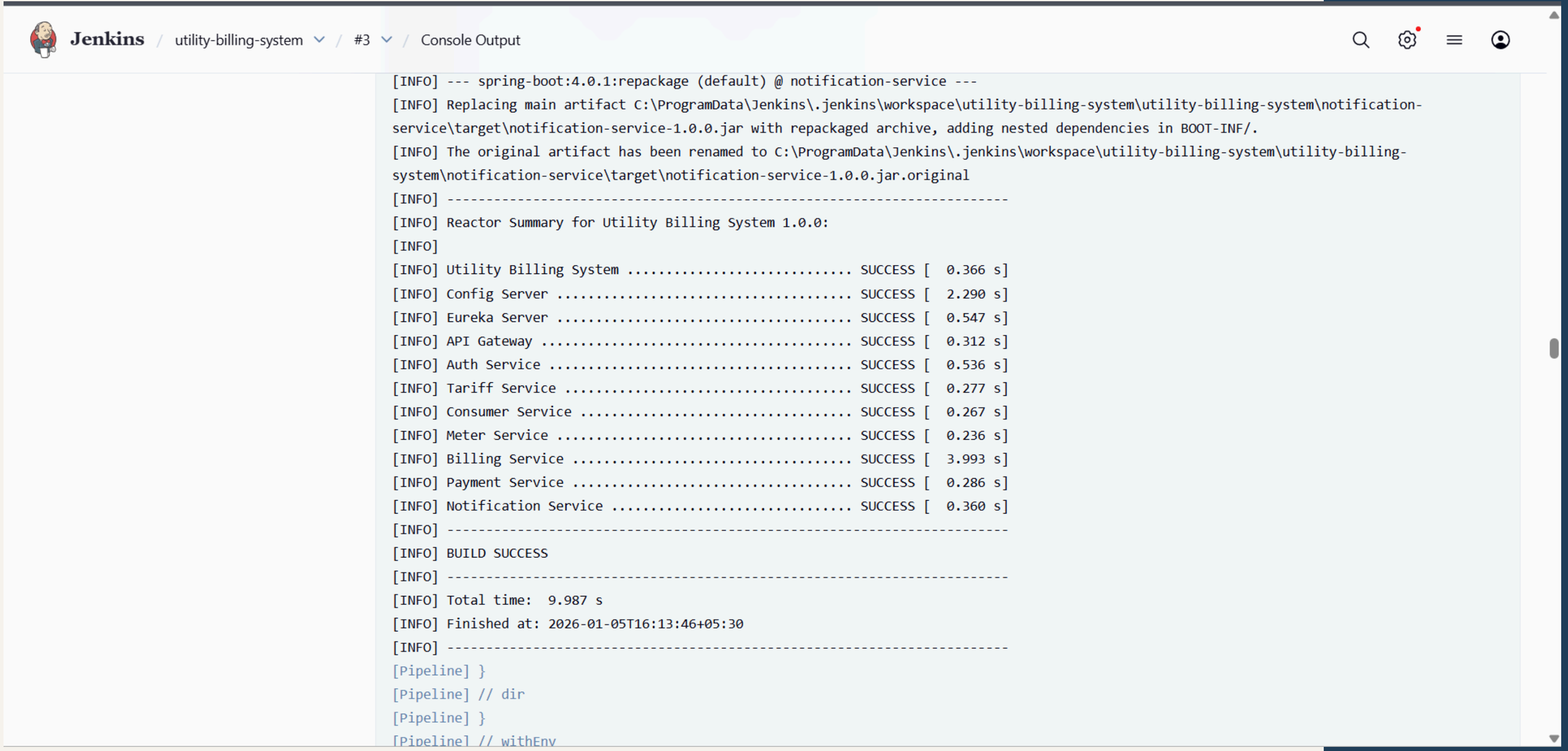
Sonarqube Report



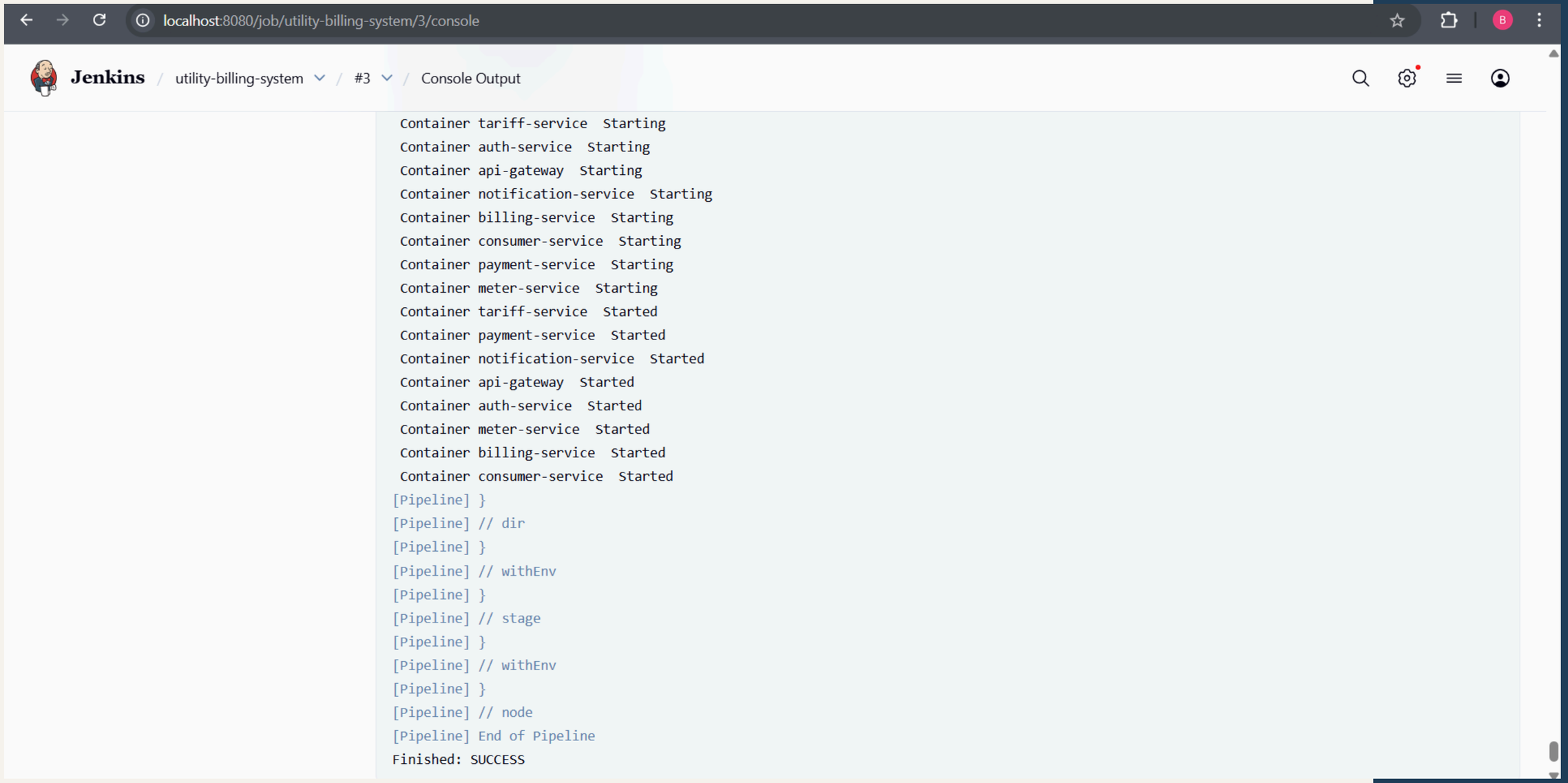
Docker


docker desktop PERSONAL
Search
Ctrl+K
Sign in
Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Containers
Give feedback
Container CPU usage
7.18% / 1200% (12 CPUs available)
Container memory usage
3.09GB / 7.42GB
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Memory usage...
Memory (%)
Disk read/
Actions
rabbitmq 2c886dd24836 rabbitmq:3- Show all ports (2) 1.55% 173.6MB / 7.59GI 2.23% 31.5MB / 7
mongo 4a18e7b977e1 mongo:7 27017:27017 0.8% 143.1MB / 7.59GI 1.84% 2.63MB / 1
config-server c20fb82b22ef utility-billin 8888:8888 0.24% 264.7MB / 7.59GI 3.4% 250KB / 6.
auth-service 1fb436f19365 utility-billin 8081:8081 0.24% 342.7MB / 7.59GI 4.41% 57.3KB / 2
notification-s bb70cc7714a9 utility-billin 8087:8087 0.36% 267.8MB / 7.59GI 3.44% 8.19KB / 2
billing-service 2d053aff1a38 utility-billin 8085:8085 0.27% 320.5MB / 7.59GI 4.12% 12.8MB / 6
tariff-service 2e72fb221488 utility-billin 8083:8083 0.29% 247.4MB / 7.59GI 3.18% 8.19KB / 2
consumer-se 8631daa10c5c utility-billin 8082:8082 0.36% 272.4MB / 7.59GI 3.5% 32.8KB / 2
meter-service c64a18375a3c utility-billin 8084:8084 0.61% 291MB / 7.59GB 3.74% 4.1KB / 25
payment-serv 019ea2cea653 utility-billin 8086:8086 0.29% 297MB / 7.59GB 3.82% 4.1KB / 24
api-gateway 94d2f82d266f utility-billin 9090:9090 0.21% 272MB / 7.59GB 3.5% 897KB / 27
Showing 13 items
Engine running
Kubernetes running RAM 7.28 GB CPU 8.77% Disk: 9.36 GB used (limit 1006.85 GB)
Terminal
Update available

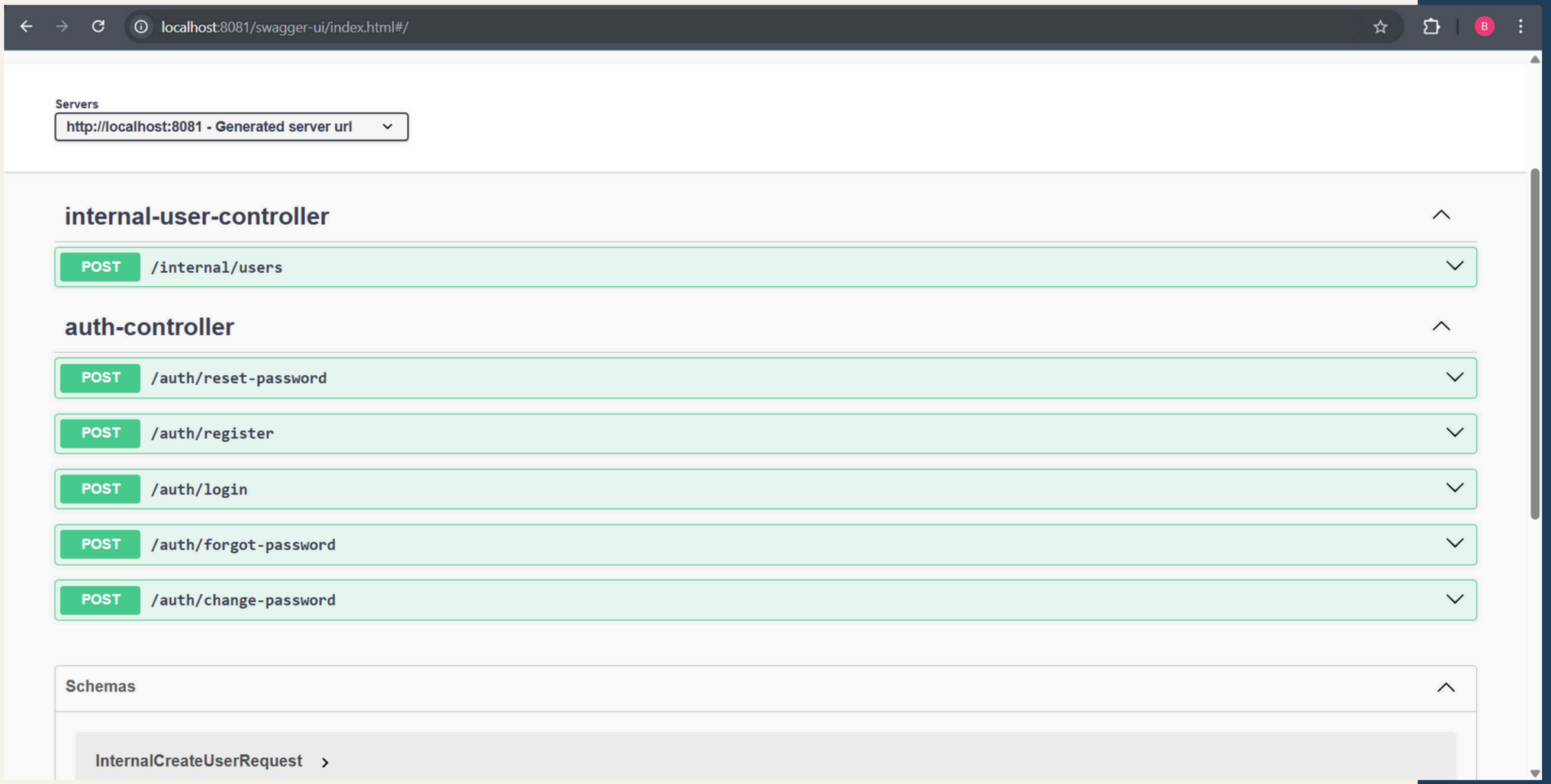
Jenkins


Jenkins / utility-billing-system / #3 / Console Output
[INFO] --- spring-boot:4.0.1:repackage (default) @ notification-service ---
[INFO] Replacing main artifact C:\ProgramData\Jenkins\.jenkins\workspace\utility-billing-system\utility-billing-system\notification-service\target\notification-service-1.0.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\ProgramData\Jenkins\.jenkins\workspace\utility-billing-system\utility-billing-system\notification-service\target\notification-service-1.0.0.jar.original
[INFO] --
[INFO] Reactor Summary for Utility Billing System 1.0.0:
[INFO]
[INFO] Utility Billing System ..... SUCCESS [0.366 s]
[INFO] Config Server ..... SUCCESS [2.290 s]
[INFO] Eureka Server ..... SUCCESS [0.547 s]
[INFO] API Gateway ..... SUCCESS [0.312 s]
[INFO] Auth Service ..... SUCCESS [0.536 s]
[INFO] Tariff Service ..... SUCCESS [0.277 s]
[INFO] Consumer Service ..... SUCCESS [0.267 s]
[INFO] Meter Service ..... SUCCESS [0.236 s]
[INFO] Billing Service ..... SUCCESS [3.993 s]
[INFO] Payment Service ..... SUCCESS [0.286 s]
[INFO] Notification Service ..... SUCCESS [0.360 s]
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 9.987 s
[INFO] Finished at: 2026-01-05T16:13:46+05:30
[INFO] --
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // withEnv

Jenkins


localhost:8080/job/utility-billing-system/3/console
Jenkins / utility-billing-system / #3 / Console Output
Container tariff-service Starting
Container auth-service Starting
Container api-gateway Starting
Container notification-service Starting
Container billing-service Starting
Container consumer-service Starting
Container payment-service Starting
Container meter-service Starting
Container tariff-service Started
Container payment-service Started
Container notification-service Started
Container api-gateway Started
Container auth-service Started
Container meter-service Started
Container billing-service Started
Container consumer-service Started
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Swagger


localhost:8081/swagger-ui/index.html#/
Servers
http://localhost:8081 - Generated server url
internal-user-controller
POST /internal/users
auth-controller
POST /auth/reset-password
POST /auth/register
POST /auth/login
POST /auth/forgot-password
POST /auth/change-password
Schemas
InternalCreateUserRequest

Conclusion

The Utility Billing System provides a modern, cloud-ready solution that eliminates manual errors and increases revenue transparency.

Future Scope:

- Mobile App: Dedicated Android/iOS app for consumers.
- Kafka Integration: Real-time event streaming for large-scale meter notifications.
- Smart Metering: IoT integration for automated reading without human intervention.

THANK YOU